

Solution Requirements

Functional & Non-Functional Requirements — Design Phase II

Date	07 March 2026
Team ID	
Project Name	Insurance Fraud Detection Using Machine Learning
Maximum Marks	4 Marks

Functional Requirements

Functional requirements define what the system must do — the specific features and capabilities that users can interact with or that the system must perform.

FR No.	Functional Requirement (Epic)	Sub-Requirement (Story / Sub-Task)
FR-1	User Registration & Authentication	Register via Email and Password Login with credentials (Email + Password) Google OAuth 2.0 Single Sign-On Password reset via OTP email Session management and JWT token handling
FR-2	Claim Submission — Automobile	Submit vehicle damage details (make, model, year, damage type) Upload claim amount and repair estimate Provide customer claim history details Receive real-time fraud probability score View top features influencing the fraud score
FR-3	Claim Submission — Health Insurance	Submit medical treatment details (treatment codes, diagnosis) Enter billing amount and healthcare provider details Provide claim frequency and policy duration Detect overbilling and duplicate claim patterns Receive fraud verdict with explanation
FR-4	Claim Submission — Property Insurance	Submit property details (type, location, value, insured amount) Enter damage report (fire, theft, natural disaster) Upload claim timing and supporting documentation Cross-verify claim value against property benchmark Flag inflated or suspicious damage claims
FR-5	Fraud Prediction Engine	ML model classifies each claim as FRAUD or GENUINE Display fraud probability score (0.0 to 1.0) Show risk category: Low / Medium / High / Critical Display top 3–5 contributing features (Explainable AI) Log prediction with timestamp and model version
FR-6	Dashboard & Analytics	View personal claim history with fraud scores Visualize fraud rate trends by domain (Auto/Health/Property) Filter claims by date range, domain, and fraud verdict Export claim reports as PDF or CSV Admin: view all claims across all users
FR-7	Notification System	Email alert sent when fraud score exceeds 0.8 threshold In-app notification banner for high-risk claims Daily summary email with fraud stats for officers Configurable threshold per domain by admin
FR-8	Admin Management Panel	Create, update, deactivate user accounts Assign roles: Claimant / Claims Officer / Risk Manager / Admin View all system claims and override fraud verdicts Monitor system performance and model accuracy metrics Update ML model configuration and retrain trigger

Non-Functional Requirements

Non-functional requirements define how the system performs — quality attributes such as security, performance, scalability, and availability.

NFR No.	Requirement	Description	Technology / Approach
NFR-1	Usability	The web interface must be intuitive and accessible for non-technical insurance officers. The UI must follow clean design principles with a maximum of 3 clicks to reach any key feature. Mobile-responsive layout required.	Bootstrap 5, Clear Navigation, Guided Onboarding Tour, Mobile-first CSS
NFR-2	Security	All data in transit must be encrypted using HTTPS/TLS 1.3. Passwords stored with bcrypt/SHA-256 hashing. Role-based access control (RBAC) enforced across all API endpoints. OWASP Top 10 vulnerabilities addressed in code review. JWT tokens expire after 24 hours.	HTTPS/TLS, bcrypt, JWT, RBAC, OWASP Guidelines, IAM Controls
NFR-3	Reliability	System must maintain 99.5% uptime. All ML model predictions and claim decisions must be logged with full auditability. Automated daily backups of database and model artefacts. Graceful error handling with user-friendly messages.	Automated Backups (IBM COS/S3), Error Handling Middleware, Health Check Endpoints
NFR-4	Performance	Fraud prediction API response time must be under 2 seconds for single claims. System must support minimum 500 concurrent users. Redis caching for frequent database queries. CDN for static frontend assets.	Flask Async, Redis Cache, CDN (Cloudflare), uWSGI/Gunicorn, Load Testing with Locust
NFR-5	Availability	Deployed on cloud with load balancer distributing traffic across minimum 2 application instances. Auto-scaling enabled for peak periods. 24x7 availability for claims processing with <1 hour recovery time objective (RTO).	Nginx Load Balancer, Cloud Auto-Scaling (IBM Cloud / AWS), Multi-AZ Deployment
NFR-6	Scalability	3-tier architecture with independently scalable UI, API, and ML layers. Micro-services-ready design using Flask Blueprints. Horizontal scaling supported via Docker and Kubernetes.	Docker, Kubernetes (future), Flask Blueprints, Stateless API Design

NFR No.	Requirement	Description	Technology / Approach
NFR-7	Maintainability	Code must follow PEP8 (Python) and ESLint (JavaScript) standards. ML pipeline must be versioned with model metadata stored in the model store. Unit test coverage > 80%. CI/CD pipeline for automated testing and deployment.	Git, GitHub Actions CI/CD, pytest, Flask-Testing, Model Versioning
NFR-8	Compliance & Audit	All claim predictions must be logged with input data, model version, score, timestamp, and reviewer decision. Audit logs must be retained for minimum 7 years per insurance regulatory requirements.	MySQL Audit Tables, Immutable Log Storage (IBM COS), Data Retention Policy